# Sudoku 1.0

**Fundamentos de la Programación II**

**Grados de la Facultad de Informática (UCM)**

Fecha máxima de entrega: 7 de abril a las 9:00

## Normas de realización de la práctica

1. La fecha límite para entregar la práctica es el **7 de abril a las 9:00**.
2. Debe entregarse a través del Campus Virtual, en la actividad Entrega práctica (versión 1).
3. Sigue los pasos que aparecen en el enunciado y ten en cuenta todas las indicaciones.
4. Aunque recomendamos que vayas haciendo la práctica de manera incremental, tal y como aparece en el enunciado, debes leerlo completamente antes de empezar.
5. En el Campus Virtual encontrarás algunos archivos para realizar pruebas.
6. No puedes añadir nuevos métodos públicos. Si está permitido añadir tantos métodos privados como consideres necesario.

*Primero resuelve el problema. Entonces, escribe el código.*

— John Johnson

## El Juego

En esta práctica vamos a explorar la implementación de un juego clásico llamado Sudoku. Según la información de wikipedia (https://es.wikipedia.org/wiki/Sudoku), es un juego matemático que se inventó a finales de la década de 1970, adquirió popularidad en Japón en la década de 1980, y se dio a conocer en el ámbito internacional en 2005, cuando numerosos periódicos empezaron a publicarlo en su sección de pasatiempos.

El Sudoku es un juego de lógica y razonamiento que habitualmente se representa a través de una cuadrícula de `9×9`, dividida en nueve subcuadrículas de `3×3`, donde cada celda de una cuadrícula puede contener elementos entre `1..9`. Puede jugarse a Sudokus de otras dimensiones, como `4x4` (subcuadrículas de `2×2`), de `6x6` (subcuadrículas de `2x3` o `3x2`), `16 x16` (subcuadrículas de `4x4`) etc. Como regla general en esta práctica, los Sudokus podrán tener dimensión $n^2xn^2$, dividido en subcuadrículas de `n x n`, donde cada cuadrícula podrá contener un elemento entre $1 \ldots n^2$.

El objetivo final del Sudoku, para el caso `9x9` (generalizable a Sudokus de otras dimensiones) es completar la cuadrícula colocando números del `1` al `9` de modo que:

- Cada fila contenga los números del `1` al `9` sin repeticiones.
- Cada columna contenga los números del `1` al `9` sin repeticiones.
- Cada subcuadrícula de `3×3` contenga los números del `1` al `9` sin repeticiones.

Un Sudoku inicial tiene algunas celdas que contienen números. Partiendo de esos números, debe deducirse dónde colocar los números que faltan hasta completar toda la cuadrícula. El Sudoku inicial se caracteriza porque tiene una única solución correcta.

Por ejemplo, en la siguiente figura, el Sudoku de la izquierda sería el Sudoku inicial con el que se empieza a jugar. Mientras que el Sudoku de la derecha es su solución.

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | | | | | | 5 | | |
| | | | | | 2 | | 8 | |
| | 5 | | | 8 | | 6 | 7 | 2 |
| | 2 | 8 | | 7 | 5 | | | |
| 7 | | 6 | 3 | 9 | | | 2 | |
| | | | 8 | | 6 | | 3 | |
| | | 1 | | 3 | | 2 | | |
| 2 | 4 | | | 1 | 8 | | | |
| | 3 | 9 | | | | | 4 | 8 |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 9 | 8 | 2 | 7 | 6 | 3 | 5 | 1 | 4 |
| 1 | 6 | 7 | 5 | 4 | 2 | 9 | 8 | 3 |
| 4 | 5 | 3 | 9 | 8 | 1 | 6 | 7 | 2 |
| 3 | 2 | 8 | 1 | 7 | 5 | 4 | 6 | 9 |
| 7 | 1 | 6 | 3 | 9 | 4 | 8 | 2 | 5 |
| 5 | 9 | 4 | 8 | 2 | 6 | 7 | 3 | 1 |
| 8 | 7 | 1 | 4 | 3 | 9 | 2 | 5 | 6 |
| 2 | 4 | 5 | 6 | 1 | 8 | 3 | 9 | 7 |
| 6 | 3 | 9 | 2 | 5 | 7 | 1 | 4 | 8 |

## Funcionalidad de la práctica

Nuestro proyecto implementará una aplicación para jugar al Sudoku, ofreciendo las siguientes opciones:

1. **Poner número:** Pide una posición `(f,c)` y un valor `v`, e intenta colocar el valor en la posición dada. Si no es posible, porque los datos de entrada son incorrectos, porque se

está intentando colocar el número en una celda no vacía, o porque `v` no es un valor válido en esa celda, entonces se manda un mensaje de error. Si se ha podido colocar el valor, puede ocurrir que algunas casillas (celdas) de la misma fila, columna o submatriz, se queden sin posibles valores para poner. A estas celdas las llamaremos **celdas bloqueadas**. Y en caso de que al poner un valor las haya, se mostrará un mensaje al usuario informándole de dicha situación.

2. **Quitar número:** Pide una posición `(f,c)` e intenta eliminar el número de esa celda. Si la operación no es posible, bien porque los datos introducidos son incorrectos o porque la celda está vacía o es una celda correspondiente al Sudoku original, entonces se manda un mensaje de error. Si se puede eliminar el valor, entonces se tendrán que actualizar aquellas celdas de la misma fila, columna o submatriz que estuvieran bloqueadas.

3. **Reiniciar:** Recupera el Sudoku original, es decir el Sudoku de partida.

4. **Posibles valores de una celda vacía:** Pide una posición `(f,c)` y devuelve los valores posibles que se pueden colocar en esa celda, cumpliendo las reglas del Sudoku. Si la posición `(f,c)` no es correcta o la celda no está vacía, entonces manda un mensaje de error.

5. **Autocompletar celdas con un valor único:** Rellena todas las celdas vacías que tienen un único valor posible.

6. **Salir:** Cierra la aplicación.

Tanto en el caso de poner un valor, como en el caso de autocompletar el Sudoku, puede ocurrir que se llegue a una situación de bloqueo. Una situación de bloqueo ocurre cuando una celda se queda sin posibles valores para colocar. Cuando la aplicación detecte una situación de bloqueo, mandará un mensaje al jugador indicando qué celdas son las que se encuentran bloqueadas. De esta forma será posible eliminar valores para desbloquear o resetear para volver al Sudoku original.

El programa comenzará solicitando el nombre de un archivo del que cargar un Sudoku original. Si la carga tiene éxito, se muestra el Sudoku y el menú correspondiente a las acciones descritas anteriormente. El jugador podrá seleccionar cualquier opción. El juego termina cuando el Sudoku ha sido resuelto, o cuando el jugador seleccione la opción `Salir`.

## Primera implementación

Para resolver la práctica debes implementar las clases que se detallan a continuación. De las clases sólo te indicamos los atributos que contienen, así como sus métodos públicos y constructoras.

### Clase Celda

Esta clase encapsula la información de una celda del Sudoku. Contendrá como atributos privados: el valor de la celda (elemento de tipo `int`) y su estado (tipo enumerado `tEstado` con los valores `ORIGINAL`, `OCUPADA`, `VACIA`). Cuando el estado de la celda es `ORIGINAL`, significa que la celda contiene un valor correspondiente al Sudoku original, y por tanto ese valor es inamovible. El estado `OCUPADA` implica que la celda contiene algún valor puesto por el

usuario. Finalmente `VACIA` hace referencia a celdas que no contienen, de momento, ningún valor y en las que se deben colocar los valores.

Las constructoras y métodos públicos que ofrece la clase se muestran a continuación. No se muestran los tipos de los parámetros, ni tampoco los tipos devueltos ya que los nombres de los métodos son suficientemente significativos.

```
Celda();
/* métodos de consulta */
es_vacia();     // devuelve true si y sólo si la celda es VACIA
es_original();  // devuelve true si y sólo si la celda es ORIGINAL
es_ocupada();   // devuelve true si y sólo si la celda es OCUPADA
dame_valor();   // devuelve el valor de la celda
/* métodos modificadores*/
set_valor(v);   // pone el valor v en la celda
set_ocupada();  // pone el estado de la celda a OCUPADA
set_original(); // pone el estado de la celda a ORIGINAL
set_vacia();    // pone el estado de la celda a VACIA
```

donde:

- La constructora por defecto pone el valor a `0` y el estado a `VACIA`.
- Los siguientes cuatro métodos son de consulta. Excepto el último, todos son booleanos y comprueban lo que su nombre indica. El método `dame_valor()` devuelve el valor de la celda.
- El resto de métodos modifican algún atributo del objeto tal y como indica su nombre. El método `set_valor(v)` pone el valor de la celda a `v`.

## Clase Tablero

Implementa un tablero de celdas a través de dos atributos privados: la dimensión real y una matriz (cuadrada) de celdas. La matriz tendrá como dimensión máxima la establecida por una constante.

Esta clase tiene una constructora por defecto que pone la dimension del tablero al tamaño máximo y una constructora con un argumento para inicializar la dimensión a un valor concreto. Como métodos públicos ofrece métodos para devolver la dimensión real de la matriz y el elemento que ocupa una posicion `(f,c)` en la matriz. Como método modificador tiene sólo uno que permite colocar una celda en la posición `(f,c)` de matriz.

## Clase ReglasSudoku

Encapsula las reglas para jugar al Sudoku. Para ello tiene como atributos privados un tablero (el tablero que almacena un Sudoku) y un contador. El contador contiene el número de celdas ocupadas (que contienen un valor) del Sudoku. Este contador se incrementa cuando se coloca un valor en una celda y se decrementa cuando se elimina el valor. Es de utilidad para comprobar si el juego ha terminado. Por otro lado llevará una *lista de posiciones bloqueadas*, es decir posiciones en las que no se puede colocar ningún valor.

Tiene una constructora por defecto que inicializa el contador de posiciones no vacías a `0` y el número de celdas bloqueadas a `0`.

Como métodos públicos tenemos:

```
/* consultoras */
dame_dimension();           // devuelve la dimensión del tablero
dame_celda(int f, int c);   // devuelve la celda en la posición (f,c)
terminado();                // true si y sólo si el Sudoku está resuelto
bloqueo();                  // true si el Sudoku tiene celdas bloqueadas
dame_num_celdas_bloqueadas(); // devuelve el número de celdas bloqueadas
dame_num_celdas_vacias();     // devuelve el número de celdas vacías
dame_celda_bloqueada(int p, int& f, int& c); // devuelve en (f,c) la celda
    bloqueada en la posición p
es_valor_posible(f,c,v); // true si y sólo si v se puede colocar en (f,c)
posibles_valores(f,c);   // devuelve el número de posibles valores para (f,c)
/* modificadoras */
pon_valor(f, c, v);   // pone v en (f,c)
quita_valor(f, c);    // pone la celda (f,c) a VACIA
reset();              // recupera el Sudoku original
autocompletar();      // rellena todas las celdas con un único valor posible
/* inicializadora */
carga_sudoku(ifstream&);   // carga un Sudoku original de un archivo
```

donde los nueve primeros métodos son de consulta. Los tres primeros son triviales. El método `bloqueo()` devuelve `true` si y sólo sí la lista de posiciones bloqueadas no es vacía. Los métodos `dame_num_celdas_bloqueadas()` `dame_num_celdas_vacias()` devuelven, respectivamente, el número de posiciones bloqueadas y de celdas vacías, y `dame_celda_bloqueada(p,f,c)` devuelve una posición `(f,c)` que se corresponde con la fila y columna del elemento que ocupa la posición `p` en la lista. Por último, el método `es_valor_posible(f,c,v)` devuelve `true` si y sólo si el valor `v` se puede colocar en la posición `(f,c)` y `posibles_valores(f,c)` devuelve el número de posibles valores que se pueden colocar en la posición `(f,c)`.

Los cuatro métodos modificadores hacen lo siguiente:

- `pon_valor(f,c,v)`: pone, si es posible, el valor `v` en la posición `(f,c)` de la matriz. Para ello comprueba que `(f,c)` y `v` son correctos, que la celda en esta posición está vacía y que `v` es un valor posible en esa celda. Si es así devuelve `true` y pone el valor en la celda, actualizando las posibles celdas bloqueadas al colocar `v`. También incrementa el valor del contador de celdas no vacías. En otro caso devuelve `false`.

- `quita_valor(f,c)`: si `(f,c)` es una posición correcta y no está vacía, pone la celda a `VACIA` y decrementa el contador de celdas con número. Actualiza además las celdas bloqueadas, ya que alguna puede dejar de estarlo. En otro caso devuelve `false`.

- `reset`: recupera el valor del Sudoku original.

- `autocompletar()`: autocompleta el tablero colocando, en todas las posiciones `(f,c)` vacías, que tienen un único valor posible `v`, el valor `v`.

- `carga_sudoku(ifstream& archivo)` carga la información de un sudoku a partir de `archivo` que ya viene abierto desde `main`. La estructura de los archivos de texto que contienen un Sudoku, es por ejemplo:

```
9
0 0 0 0 0 0 5 0 0
0 0 0 0 0 2 0 8 0
0 5 0 0 8 0 6 7 2
0 2 8 0 7 5 0 0 0
7 0 6 3 9 0 0 2 0
0 0 0 8 0 6 0 3 0
0 0 1 0 3 0 2 0 0
2 4 0 0 1 8 0 0 0
0 3 9 0 0 0 0 4 8
```

donde la primera línea contiene el tamaño del Sudoku, en este caso `9x9`, y el resto de líneas representa la información de cada fila del Sudoku. El valor `0` representa una celda cuyo estado es `VACIA`. Los valores distintos de cero se corresponden con celdas cuyo estado es `ORIGINAL`. El fichero anterior se corresponde con el Sudoku que aparece en la descripción del juego.

## Función main

La función `main` carga un Sudoku de un archivo de texto, y si la apertura es correcta muestra el Sudoku original junto con el siguiente menú:

Una vez que se introduce una opción correcta, se ejecuta la opción tal y como se ha explicado en la descripción del juego. Por ejemplo:

```
Elige una opcion: 4
Fila y columna entre 1...9: 1 1
Los valores posibles para la celda son: { 1 3 4 6 8 9 }

    1  2  3   4  5  6   7  8  9
  ┼──────────┼──────────┼──────────┼
1 │ ·  ·  ·  │ ·  ·  ·  │ 5  ·  ·  │
2 │ ·  ·  ·  │ ·  ·  2  │ ·  8  ·  │
3 │ ·  5  ·  │ ·  8  ·  │ 6  7  2  │
  ├──────────┼──────────┼──────────┼
4 │ ·  2  8  │ ·  7  5  │ ·  ·  ·  │
5 │ 7  ·  6  │ 3  9  ·  │ ·  2  ·  │
6 │ ·  ·  ·  │ 8  ·  6  │ ·  3  ·  │
  ├──────────┼──────────┼──────────┼
7 │ ·  ·  1  │ ·  3  ·  │ 2  ·  ·  │
8 │ 2  4  ·  │ ·  1  8  │ ·  ·  ·  │
9 │ ·  3  9  │ ·  ·  ·  │ ·  4  8  │
  ┼──────────┼──────────┼──────────┼
```

El juego termina cuando se elige la opción `salir` o se termina el Sudoku. Ten en cuenta que se puede llegar a situaciones de bloqueo, en las que ya no se puede continuar. Eso ocurre cuando alguna celda vacía se queda sin posibles valores. En tal caso la aplicación te avisa de este hecho, y puedes recolocar valores utilizando la opción `quitar número`. Por ejemplo, si en el sudoku anterior llegamos a la situación:

```
 1.- poner numero
 2.- quitar numero
 3.- reset
 4.- posibles valores de una celda vacia
 5.- autocompletar celdas con valor unico
 6.- salir
Elige una opcion: 1
Fila y columna entre 1...9: 7 2
Valor: 6
Sudoku bloqueado.....Las casillas bloqueadas son: (7,8)  (9,1)


    1  2  3   4  5  6   7  8  9
  ┼──────────┼──────────┼──────────┼
1 │ ·  ·  ·  │ ·  ·  ·  │ 5  ·  ·  │
2 │ ·  ·  ·  │ ·  ·  2  │ ·  8  ·  │
3 │ ·  5  ·  │ ·  8  ·  │ 6  7  2  │
  ├──────────┼──────────┼──────────┼
4 │ ·  2  8  │ 1  7  5  │ ·  ·  ·  │
5 │ 7  1  6  │ 3  9  4  │ 8  2  5  │
6 │ ·  9  ·  │ 8  2  6  │ ·  3  ·  │
  ├──────────┼──────────┼──────────┼
7 │ 5  6  1  │ ·  3  9  │ 2  ·  ·  │
8 │ 2  4  ·  │ ·  1  8  │ ·  ·  ·  │
9 │ ·  3  9  │ ·  ·  7  │ 1  4  8  │
  ┼──────────┼──────────┼──────────┼
```

entonces no tiene sentido continuar, dado que hay dos posiciones bloqueadas. Por lo tanto deben quitarse valores o hacer un `reset`.

Junto con el enunciado aparece un fichero "`colors.h`" con distintas constantes para poner colores. Para activar un color al escribir, basta poner `cout << ORANGE`. El texto a continuación saldrá naranja. Si queremos volver al color normal podemos utilizar `cout << RESET`.

## Mejoras en la implementación

Como habrás observado al realizar la primera implementación, se llevan a cabo un gran número de búsquedas en la matriz, que se podrían evitar almacenando las relaciones entre las celdas del Sudoku. Es decir, si en una posición `(f,c)` se ha colocado un valor `v`, podemos almacenar que todas las celdas de la fila `f`, columna `c` y de la submatriz correspondientes a `(f,c)`, no pueden tener como valor `v`. Como además puede ocurrir que alguna de esas celdas esté afectada por `v` por otra celda distinta de `(f,c)` que contenga también `v`, necesitamos almacenar, para las posiciones no posibles, el número de celdas que la afectan.

Por ejemplo, si nos fijamos en la matriz que aparece a continuación:

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 8 | · | · | · | · | · | 5 | · | · |
| 2 | · | · | · | · | · | 2 | · | 8 | · |
| 3 | · | 5 | · | · | 8 | · | 6 | 7 | 2 |
| 4 | · | 2 | 8 | · | 7 | 5 | · | · | · |
| 5 | 7 | · | 6 | 3 | 9 | · | · | 2 | · |
| 6 | · | · | · | 8 | · | 6 | · | 3 | · |
| 7 | · | · | 1 | · | 3 | · | 2 | · | · |
| 8 | 2 | 4 | · | · | 1 | 8 | · | · | · |
| 9 | · | 3 | 9 | · | · | · | · | 4 | 8 |

se observa que la posición `(3,3)` está afectada por el valor `8` de tres formas distintas. Una que proviene de la submatriz, otra que proviene de la fila `3` y otra que proviene de la columna `3`. Por lo tanto, aunque eliminaramos el valor `8` dela posición `(1,1)`, la celda `(3,3)` seguiría afectada por dos celdas y el valor `8` no podría ser un valor posible.

Para poder almacenar esta información, vamos a utilizar una matriz `3D`, que tendrá la siguente forma:

```
typedef struct {
  bool posible;
  int celdas_que_afectan;
} tValor;

typedef struct {
  int nFilas;
  int nColumnas;
  tValor valores[DIM][DIM][MAX_VALORES];
} tValores;

tValores valores_celda;
```

donde `valores_celda.valores[f][c][v].posible = true` significa que el valor `v` se puede colocar en la posición `(f,c)`. En este caso el campo `celdas_que_afectan` será `0` para esa posición. Por el contario, si `valores_celda.valores[f][c][v].posible = false` implica que el valor `v` no se puede poner en esa posición, y en ese caso `celdas_que_afectan` será el número de celdas que afectan al valor `v` de esa posición.

Para incorporar esta mejora, excepto la implementación de la clase `ReglasSudoku`, no debes modificar absolutamente nada más de la práctica.